CERTIK

YFSX.VIN

Security Assessment

CertiK Assessed on Apr 10th, 2026

CertiK Assessed on Apr 10th, 2026

YFSX.VIN

The security assessment was prepared by CertiK.

Executive Summary

TYPES	ECOSYSTEM	METHODS
Deflationary Token	Binance Smart Chain (BSC)	Formal Verification, Manual Review, Static Analysis

LANGUAGE	TIMELINE
Solidity	Preliminary comments published on 04/09/2026 Final report published on 04/10/2026

Vulnerability Summary

20	2	0	18	0
Total Findings	Resolved	Partially Resolved	Acknowledged	Declined

3	Centralization	2 Renounced, 1 Acknowledged	Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.
0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
3	Major	3 Acknowledged	Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.
9	Medium	9 Acknowledged	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
5	Minor	5 Acknowledged	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
0	Informational		Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | YFSX.VIN

Audit Summary

Executive Summary

Vulnerability Summary

Codebase

Audit Scope

Approach & Methods

Findings

YFS-01 : Initial Token Distribution

YFS-02 : Centralized Risk In `addLiquidity`

YFS-03 : Centralization Related Risk

YFS-04 : Double Accounting Of Burned Tokens

YFS-05 : Broken Token Invariant Destroys Dividend Math

YFS-06 : One-Sided AMM Token Injection Deflates Pool Price

YFS-07 : Ownership Can Be Regained After Renouncement

YFS-08 : Potential Sandwich Attacks

YFS-09 : Phantom Balance And `includeInReward` Logic Flaw Enables Token Minting And Massive Dilution

YFS-10 : The Auto-Swap Threshold Is Larger Than The Entire Universe

YFS-11 : Anti-Whale Limit Is Cannot Be Used

YFS-12 : Router Should Not Be In The Fee Exempt Whitelist

YFS-13 : Out Of Gas Exceptions In `difidendToHolders` Loop

YFS-14 : Massive Tax Evasion Via Alternative Pools

YFS-15 : Rate Reset And Balance Wipeout Via Deliver Function

YFS-16 : Third-Party Dependency Usage

YFS-17 : No Cap On Fees

YFS-18 : Zero-Value Transfer Forces Swap Execution

YFS-19 : Uncallable Setters Due To Private Visibility

YFS-20 : Stale Fee Cache In Fee-Exempt Transfers Re-Enables Global Fees After Zero-Fee Mode

Optimizations

YFS-21 : Lack of Array Bounds on Excluded Accounts

YFS-22 : `amountToStopBurn` is a Ghost Variable

Appendix

Disclaimer

CODEBASE | YFSX.VIN

Repository

https://bscscan.com/address/0x85e43bf8faaf04ceddcd03d6c07438b72606a988

https://bscscan.com/address/0xb7ec60cf8ef96ed48b119277bc7a954a87f27388

Commit

0x85e43bf8faaf04ceddcd03d6c07438b72606a988

0xb7ec60cf8ef96ed48b119277bc7a954a87f27388

AUDIT SCOPE | YFSX.VIN

mainnet
VulgarTycoon.sol
YfsxCoin.sol

APPROACH & METHODS | YFSX.VIN

This audit was conducted for YFSX.VIN to evaluate the security and correctness of the smart contracts associated with the YFSX.VIN project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Formal Verification, Manual Review, and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

FINDINGS | YFSX.VIN

20	0	3	3	9	5	0
Total Findings	Critical	Centralization	Major	Medium	Minor	Informational

This report has been prepared for YFSX.VIN to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 20 issues were identified. Leveraging a combination of Formal Verification, Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
YFS-01	**Initial Token Distribution**	**Centralization**	**Centralization**	**Acknowledged**
YFS-02	**Centralized Risk In `addLiquidity`**	**Centralization**	**Centralization**	**Renounced**
YFS-03	**Centralization Related Risk**	**Centralization**	**Centralization**	**Renounced**
YFS-04	Double Accounting Of Burned Tokens	Financial Manipulation	Major	Acknowledged
YFS-05	Broken Token Invariant Destroys Dividend Math	Financial Manipulation	Major	Acknowledged
YFS-06	One-Sided AMM Token Injection Deflates Pool Price	Financial Manipulation	Major	Acknowledged
YFS-07	Ownership Can Be Regained After Renouncement	Logical Issue	Medium	Acknowledged
YFS-08	Potential Sandwich Attacks	Logical Issue	Medium	Acknowledged
YFS-09	Phantom Balance And `includeInReward` Logic Flaw Enables Token Minting And Massive Dilution	Financial Manipulation	Medium	Acknowledged
YFS-10	The Auto-Swap Threshold Is Larger Than The Entire Universe	Design Issue	Medium	Acknowledged

ID	Title	Category	Severity	Status
YFS-11	Anti-Whale Limit Is Cannot Be Used	Design Issue	Medium	Acknowledged
YFS-12	Router Should Not Be In The Fee Exempt Whitelist	Coding Issue	Medium	Acknowledged
YFS-13	Out Of Gas Exceptions In `difidendToHolders` Loop	Volatile Code	Medium	Acknowledged
YFS-14	Massive Tax Evasion Via Alternative Pools	Financial Manipulation	Medium	Acknowledged
YFS-15	Rate Reset And Balance Wipeout Via Deliver Function	Volatile Code	Medium	Acknowledged
YFS-16	Third-Party Dependency Usage	Design Issue	Minor	Acknowledged
YFS-17	No Cap On Fees	Logical Issue	Minor	Acknowledged
YFS-18	Zero-Value Transfer Forces Swap Execution	Design Issue	Minor	Acknowledged
YFS-19	Uncallable Setters Due To Private Visibility	Coding Issue	Minor	Acknowledged
YFS-20	Stale Fee Cache In Fee-Exempt Transfers Re-Enables Global Fees After Zero-Fee Mode	Volatile Code	Minor	Acknowledged

YFS-01 | Initial Token Distribution

Category	Severity	Location	Status
Centralization	**Centralization**	**YfsxCoin.sol: 486**	**Acknowledged**

Description

All of the `YFSX` tokens are sent to the contract deployer. This is a centralization risk because the deployer can distribute tokens without obtaining the consensus of the community. Any compromise to these addresses may allow a hacker to steal and sell tokens on the market, resulting in severe damage to the project.

Recommendation

It is recommended that the team be transparent regarding the initial token distribution process. The token distribution plan should be published in a public location that the community can access. The team should make efforts to restrict access to the private keys of the deployer account or EOAs. A multi-signature (⅔, ⅗) wallet can be used to prevent a single point of failure due to a private key compromise. Additionally, the team can lock up a portion of tokens, release them with a vesting schedule for long-term success, and deanonymize the project team with a third-party KYC provider to create greater accountability.

Alleviation

[YFSX.VIN, 04/10/2026]: The team acknowledged this issue.

[CertiK, 04/10/2026]: It is suggested to implement the aforementioned methods to avoid centralized failure. Also, CertiK strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

YFS-02 | Centralized Risk In `addLiquidity`

Category	Severity	Location	Status
Centralization	**Centralization**	**YfsxCoin.sol: 726~731**	**Renounced**

Description

The `addLiquidity` function calls the `uniswapV2Router.addLiquidityETH` function with the `to` address specified as `owner()` for acquiring the generated LP tokens from the `YFSX-WBNB` pool. As a result, over time the `_owner` address will accumulate a significant portion of LP tokens. If the `_owner` is an EOA (Externally Owned Account), mishandling of its private key can have devastating consequences to the project as a whole.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations; AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised; AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations; AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement. AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles. OR
- Remove the risky functionality.

Alleviation

[CertiK, 04/10/2026]: The owner role of the `YfsxCoin` contract deployed at address https://bscscan.com/address/0xb7ec60cf8ef96ed48b119277bc7a954a87f27388#code has been renounced to the zero address.

Renounce transaction:
https://bscscan.com/tx/0x0f04a3df6bfbde720795ecc2129c72caa025f98fa28e5b117620788951b55a54

YFS-03 | Centralization Related Risk

Category	Severity	Location	Status
Centralization	Centralization	VulgarTycoon.sol: 211; YfsxCoin.sol: 415	Renounced

Description

In the contract `YfsxCoin` the role `owner` has authority over the functions shown below. Any compromise to the `owner` account may allow the hacker to take advantage of this authority.

- renounceOwnership(): The owner gives up control of the contract forever.
- transferOwnership(newOwner): Transfers control of the contract to a different wallet address.
- lock(time) / unlock(): Allows the owner to temporarily lock themselves out of the contract for a specific period.
- setTaxFeePercent(taxFee): Changes the percentage of each transaction redistributed to existing holders.
- setLiquidityFeePercent(liquidityFee): Changes the percentage of each transaction taken for liquidity/marketing.
- setMarketingDivisor(divisor): Adjusts how the collected fees are split between the marketing wallet and other functions.
- setMarketingAddress(address): Changes which wallet receives the marketing funds generated by trade taxes.
- excludeFromFee(account): Exempts a specific address from paying any transaction taxes.
- includeInFee(account): Re-enables taxes for a previously excluded address.
- excludeFromReward(account): Prevents an address from receiving "reflections" dividends.
- includeInReward(account): Allows an excluded address to start receiving dividends again.
- setMaxTxAmount(maxTxAmount): Sets the maximum number of tokens allowed in a single transaction.
- setNumTokensSellToAddToLiquidity(amount): Sets the threshold of tokens the contract.
- setBuybackUpperLimit(limit): Sets the maximum amount of ETH the contract.
- setSwapAndLiquifyEnabled(bool): Turns the automatic "sell-to-add-liquidity" feature on or off.
- setBuyBackEnabled(bool): Turns the automatic "buy-back-and-burn" feature on or off.
- prepareForPreSale(): A function that instantly disables taxes and raises limits.
- afterPreSale(): A function used after the launch to instantly set the taxes.

In the contract `VulgarTycoon` the role `owner` has authority over the functions shown below. Any compromise to the `owner` account may allow the hacker to take advantage of this authority.

- setBuyFeeToBurn(value): Adjusts the % of tokens permanently destroyed when someone buys.
- setSellFeeToBurn(value): Adjusts the % of tokens burned when someone sells.
- setBuyFeeToDistributor(value): Changes the % of buy taxes sent to the marketing/distributor wallet.
- setSellFeeToDistributor(value): Changes the % of sell taxes sent to the marketing/distributor wallet.
- setDistributorAddr(account): Updates the wallet address that receives the marketing/distributor fees.
- exemptFromFee(account, flag): Whitelists or removes an address from the tax system.

- setAmountToStopBurn(value): Sets the total supply level at which the burning mechanism should theoretically stop.
- setAmountTodifidend(value): Sets the minimum balance a user must hold to be eligible for holder dividends.
- setAmountToLPDifidend(value): Sets the "threshold" amount of tokens the contract must collect before it triggers a swap for ETH to pay LP dividends.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR

- Remove the risky functionality.

Alleviation

[CertiK, 04/10/2026]: The owner role of the `YfsxCoin` contract deployed at address https://bscscan.com/address/0xb7ec60cf8ef96ed48b119277bc7a954a87f27388 has been renounced to the zero address.

Renounce transaction:
https://bscscan.com/tx/0x0f04a3df6bfbde720795ecc2129c72caa025f98fa28e5b117620788951b55a54

The owner role of the `VulgarTycoon` contract deployed at address https://bscscan.com/address/0x85e43bf8faaf04ceddcd03d6c07438b72606a988 has been renounced to the zero address.

Renounce transaction:
https://bscscan.com/tx/0x1d76aeb23f987fcf509e1168a7d5fb6711effbae88d30108ac46c4a965b6a28e

YFS-04 | Double Accounting Of Burned Tokens

Category	Severity	Location	Status
Financial Manipulation	Major	VulgarTycoon.sol: 320~322, 336~338	Acknowledged

Description

The `VulgarTycoon` contract implements a deflationary tax mechanism intended to burn 1% of tokens on buys, sells, and normal transfers. This burn mechanism is handled inside the `processFee` function for DEX trades, and directly inside `_transfer` for wallet-to-wallet transfers.

The contract simultaneously implements two different methods of burning tokens: it adds the tax amount to the dead address's balance AND it subtracts the tax amount from the global `_totalSupply` variable. This "Double Dip" in accounting artificially deflates the reported total supply while the physical tokens remain inside the contract's balances. Whenever a taxable transfer occurs, the contract executes the following logic to handle the burn:

```
uint burnAmount = amount.mul(feeToBurn).div(100);
_balances[dead] = _balances[dead].add(burnAmount);
_totalSupply = _totalSupply.sub(burnAmount);
```

In standard ERC20/BEP20 implementations, a token burn must either be handled by sending tokens to an inaccessible dead address (leaving `_totalSupply` untouched) OR by erasing the tokens from existence by subtracting them from balances and `_totalSupply`. By doing both, the contract credits 1 token to the dead wallet, but also drops the `_totalSupply` counter by 1.

This causes severe accounting discrepancies. The token supply metrics reported to block explorers and DEXs will be fundamentally incorrect, as the contract claims to have a smaller total supply than the sum of the tokens it is currently tracking in its internal ledger.

Proof of Concept

```
function testDoubleAccountingBurnInProcessFee() public {
    uint256 deadBalanceBefore = token.balanceOf(token.dead());
    uint256 totalSupplyBefore = token.totalSupply();

    console.log("=== BEFORE SELL (Whole Tokens) ===");
    console.log("Dead Address Balance: ", deadBalanceBefore / 10**18);
    console.log("Total Supply:         ", totalSupplyBefore / 10**18);

    uint256 transferAmount = 1000 * 10**18;

    vm.prank(alice);
    token.transfer(pair, transferAmount);

    uint256 deadBalanceAfter = token.balanceOf(token.dead());
    uint256 totalSupplyAfter = token.totalSupply();
    uint256 burnAmount = transferAmount / 100;

    console.log("=== AFTER SELL Hitting processFee (Whole Tokens) ===");
    console.log("Dead Address Balance: ", deadBalanceAfter / 10**18);
    console.log("Total Supply:         ", totalSupplyAfter / 10**18);

    console.log("=== ACCOUNTING DISCREPANCY ===");
    console.log("Tokens added to Dead Vault: ", (deadBalanceAfter -
deadBalanceBefore) / 10**18);
    console.log("Tokens erased from Supply:  ", (totalSupplyBefore -
totalSupplyAfter) / 10**18);

    // Prove that the burn was applied twice (once to dead address, once to total
supply)
    assertEq(deadBalanceAfter, deadBalanceBefore + burnAmount, "Dead balance should
increase");
    assertEq(totalSupplyAfter, totalSupplyBefore - burnAmount, "Supply should
decrease");
}
```

Recommendation

Choose a single burn mechanism and remove the other. Either remove `_totalSupply = _totalSupply.sub(burnAmount);` to rely solely on the dead address balance, OR remove `_balances[dead] = _balances[dead].add(burnAmount);` to implement true deflation.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

YFS-05 | Broken Token Invariant Destroys Dividend Math

Category	Severity	Location	Status
Financial Manipulation	Major	VulgarTycoon.sol: 320~322, 336~338	Acknowledged

Description

The contract distributes a percentage of the transfer amount as a dividend to existing token holders when `feeTodifidend` is active. The distribution is calculated proportionally based on the user's balance and the total supply: `difidendAmount = amount.mul(holderAmount).div(_totalSupply)`.

Because the double-accounting burn bug reduces `_totalSupply` without actually removing tokens from the `_balances` mapping, the fundamental invariant (`Sum of Balances == Total Supply`) is permanently broken. This continuously shrinks the denominator in the dividend calculation, which will inevitably cause the contract to attempt to distribute more dividend tokens than it actually collected.

When a user trades 1,000 tokens, 10 tokens are sent to the dead address, 10 to the pair, 10 to the contract, and 10 to the distributor, while the recipient gets 960. The net change across all `_balances` is exactly 0. However, `_totalSupply` is reduced by 10. Because `_totalSupply` shrinks on every trade but the actual balances (the `holderAmount` numerators) do not shrink proportionally, the resulting fraction `(holderAmount / _totalSupply)` becomes artificially inflated.

As trading volume continues, `_totalSupply` will eventually drop low enough that the sum of the dividend payouts calculated by this formula will exceed the actual dividend fee pool available. When this mathematical tipping point is reached, the `_balances[tokenHolders[i]].add(difidendAmount)` loop will attempt to pull tokens that don't exist.

Proof of Concept

function testProcessFeeBreaksInvariant() public { // The invariant for an ERC20/BEP20 token is: // Sum of all balances == Total Supply

```
    uint256 aliceBalBefore = token.balanceOf(alice);
    uint256 pairBalBefore = token.balanceOf(pair);
    uint256 deadBalBefore = token.balanceOf(token.dead());
    uint256 contractBalBefore = token.balanceOf(address(token));
    uint256 totalSupplyBefore = token.totalSupply();

    uint256 transferAmount = 1000 * 10**18;

    vm.prank(alice);
    token.transfer(pair, transferAmount);

    uint256 aliceBalAfter = token.balanceOf(alice);
    uint256 pairBalAfter = token.balanceOf(pair);
    uint256 deadBalAfter = token.balanceOf(token.dead());
    uint256 contractBalAfter = token.balanceOf(address(token));
    uint256 totalSupplyAfter = token.totalSupply();

    int256 aliceChange = int256(aliceBalAfter) - int256(aliceBalBefore);
    int256 pairChange = int256(pairBalAfter) - int256(pairBalBefore);
    int256 deadChange = int256(deadBalAfter) - int256(deadBalBefore);
    int256 contractChange = int256(contractBalAfter) - int256(contractBalBefore);

    int256 totalBalanceChange = aliceChange + pairChange + deadChange +
contractChange;
    int256 totalSupplyChange = int256(totalSupplyAfter) - int256(totalSupplyBefore);

    console.log("Tracking exactly where the 1,000 tokens went:");

    if (aliceChange < 0) {
        console.log("Alice Balance Change:      -", uint256(-aliceChange) / 10**18);
    } else {
        console.log("Alice Balance Change:      +", uint256(aliceChange) / 10**18);
    }
    console.log("Pair Balance Change:       +", uint256(pairChange) / 10**18);
    console.log("Dead Balance Change:       +", uint256(deadChange) / 10**18);
    console.log("Contract Balance Change:   +", uint256(contractChange) / 10**18);

    if (totalBalanceChange < 0) {
        console.log("Net Change in Balances:    -", uint256(-totalBalanceChange) /
10**18);
    } else if (totalBalanceChange == 0) {
        console.log("Net Change in Balances:     0");
    } else {
        console.log("Net Change in Balances:    +", uint256(totalBalanceChange) /
10**18);
    }

    console.log("Tracking what happened to the Total Supply variable:");
```

```
    if (totalSupplyChange < 0) {
        console.log("Net Change in Total Supply:-", uint256(-totalSupplyChange) /
10**18);
    } else {
        console.log("Net Change in Total Supply:+", uint256(totalSupplyChange) /
10**18);
    }

    assertEq(totalBalanceChange, 0, "Sum of balances did not change");
    assertEq(totalSupplyChange, -int256(transferAmount / 100), "Total supply
decreased");
}
```

Recommendation

Fixing the Double Accounting Burn bug (by ensuring `_totalSupply` is only modified when tokens are actually erased from `_balances`) will restore the invariant and protect the mathematical integrity of the dividend distribution formula.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

YFS-06 | One-Sided AMM Token Injection Deflates Pool Price

Category	Severity	Location	Status
Financial Manipulation	Major	VulgarTycoon.sol: 340~342	Acknowledged

Description

The contract taxes transfers to fund the liquidity pool (`feeToLiquidity`). The intention of liquidity taxes is to pair the taxed tokens with BNB and add them as deep liquidity to the PancakeSwap router, stabilizing the token's price.

The contract implements the liquidity fee by sending the taxed tokens directly to the PancakeSwap pair address without pairing them with BNB or calling `sync()` . This inflates the token reserve on the AMM without adding corresponding BNB, forcing the PancakeSwap pricing algorithm to devalue the token on every trade.

Instead of accumulating tokens and swapping half for BNB to provide two-sided liquidity via the router, the contract does this:

```
uint addLiquidityAmount = amount.mul(feeToLiquidity).div(100);
_balances[_pairAddress] = _balances[_pairAddress].add(addLiquidityAmount);
```

Automated Market Makers (AMMs) like PancakeSwap determine price using the `x * y = k` constant product formula. When tokens are force-fed into the pair's address without corresponding BNB, the pool's token reserve becomes artificially inflated. Upon the next normal trade (which triggers a sync of the reserves), the AMM registers a higher supply of VIN relative to BNB, meaning the price of 1 VIN drops.

This creates a permanent, invisible sell pressure on the token. Every single transaction drains value from the token price into the void. It does not provide actual liquidity depth, it simply guarantees that the chart will trend downward over time, resulting in continuous financial loss for holders.

Scenario

Imagine the PancakeSwap pool currently holds exactly 100 VIN and 10 BNB. According to the math of Automated Market Makers (AMMs), the price of 1 VIN is currently 0.1 BNB (10 / 100).

1. Alice buys some tokens, and the 1% "Liquidity Fee" takes 10 VIN from her.
2. The contract dumps those 10 VIN directly into the pool.
3. Now the pool has 110 VIN and 10 BNB.
4. The next time anyone makes a trade, PancakeSwap recalculates the price based on the new balances.
5. The new price of 1 VIN is now 0.09 BNB (10 / 110).

Recommendation

Refactor the liquidity tax logic. The contract should accumulate the liquidity fee tokens internally. Once a threshold is reached, it should swap 50% of the accumulated tokens for BNB via the router, and then call `_router.addLiquidity` with

the remaining 50% VIN and the acquired BNB to mint LP tokens securely.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

YFS-07 | Ownership Can Be Regained After Renouncement

Category	Severity	Location	Status
Logical Issue	Medium	YfsxCoin.sol: 189, 196~201	Acknowledged

Description

After ownership is renounced, the potential for the original owner to regain ownership exists due to the contract code not resetting the state variable for the new owner candidate to a default value.

This situation arises because the function to renounce ownership does not properly clear or update the related state variable. This allows the original owner to exploit this vulnerability under certain conditions, bypassing security checks and regaining ownership of the contract by calling the relevant function again.

Recommendation

To mitigate this issue, it is recommended to clear the state variable to avoid ownership regain after renouncement.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

YFS-08 | Potential Sandwich Attacks

Category	Severity	Location	Status
Logical Issue	Medium	YfsxCoin.sol: 695, 712	Acknowledged

Description

A sandwich attack can occur when an attacker notices a transaction involving token swaps or liquidity additions that lacks slippage restrictions or minimum output amount settings. The attacker can exploit this by frontrunning the transaction to purchase one of the assets, thereby manipulating the exchange rate, and then backrunning the transaction to sell the asset for a profit.

The referenced functions are invoked without setting slippage restrictions or minimum output amounts, making transactions that trigger these functions particularly vulnerable to sandwich attacks, especially when large input amounts are involved.

Recommendation

We recommend setting reasonable minimum output amounts, instead of 0, based on token prices when calling the aforementioned functions. If the price cannot be determined during the swap (e.g., not available from an external price oracle such as Chainlink), it's recommended to limit the swap amount in one transaction.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

YFS-09 Phantom Balance And `includeInReward` Logic Flaw Enables Token Minting And Massive Dilution

Category	Severity	Location	Status
Financial Manipulation	Medium	YfsxCoin.sol	Acknowledged

Description

The contract uses a dual-accounting system (`t-space` for real tokens, `r-space` for reflections) to distribute taxes to holders. The owner can exclude wallets from earning reflections and re-include them later using `includeInReward()` . Excluded wallets have their real token balance locked into `_tOwned` , but still maintain a synchronized `_rOwned` balance.

When an excluded wallet transfers tokens out after the global reflection rate has decayed, the contract deducts too little from its `_rOwned` balance, leaving a "phantom" balance even when the wallet is empty. If the owner ever re-includes this wallet into rewards via `includeInReward()` , this phantom balance is not properly adjusted. This math failure instantly converts internal numbers into millions of newly minted, spendable tokens for the wallet, while simultaneously spiking the global reflection rate and permanently diluting every other innocent holder's real token balance.

The root cause stems from how `_transferFromExcluded` calculates the reflection amount to deduct:

```
uint256 rAmount = tAmount.mul(currentRate);
_tOwned[sender] = _tOwned[sender].sub(tAmount);
_rOwned[sender] = _rOwned[sender].sub(rAmount);
```

Because the `currentRate` drops steadily over time, the `rAmount` deducted is much smaller than the amount originally added when the tokens entered the wallet. When the wallet is emptied, `_tOwned` hits 0, but `_rOwned` remains massive (a "phantom" balance).

When `includeInReward()` is later called on this account, the contract executes:

```
_tOwned[account] = 0;
_isExcluded[account] = false;
```

This causes two catastrophic failures:

1. **Token Minting:** Because the account is no longer excluded, its public balance is calculated purely by `tokenFromReflection(_rOwned[account])`. The phantom `_rOwned` instantly inflates the user's balance from zero to thousands or millions of tokens that never actually existed.
2. **Global Dilution:** `_getCurrentSupply()` recalculates the global rate by no longer subtracting this massive `_rOwned` balance. This mathematical breakage causes the global rate to artificially spike, immediately yielding fewer real tokens for every other user.

A single transaction can print massive amounts of tokens out of thin air, destroying the token's value and draining DEX liquidity. Simultaneously, innocent users permanently lose a significant percentage of their token balance simply by holding, with no warning and no recourse.

Proof of Concept

function testIncludeInRewardMoneyPrinter() public { console.log("=== POC 1: INCLUDE IN REWARD MONEY PRINTER ===");

```
    vm.prank(owner);
    token.excludeFromReward(alice);

    uint256 aliceStart = token.balanceOf(alice);
    console.log("Alice balance after exclusion:", aliceStart / ONE_TOKEN);

    // Generate reflection volume to decay the rate
    vm.startPrank(bob);
    token.transfer(charlie, 500 * ONE_TOKEN);
    token.transfer(charlie, 200 * ONE_TOKEN);
    vm.stopPrank();

    vm.startPrank(charlie);
    token.transfer(bob, 500 * ONE_TOKEN);
    token.transfer(bob, 200 * ONE_TOKEN);
    vm.stopPrank();

    // Alice transfers most of her tokens out at the new lower rate
    vm.prank(alice);
    token.transfer(address(0x105), 900 * ONE_TOKEN);

    uint256 aliceAfterTransfer = token.balanceOf(alice);
    console.log("Alice balance after sending 900 tokens:", aliceAfterTransfer /
ONE_TOKEN);

    // Re-include Alice
    vm.prank(owner);
    token.includeInReward(alice);

    uint256 aliceAfterInclude = token.balanceOf(alice);
    console.log("Alice balance after re-inclusion:", aliceAfterInclude / ONE_TOKEN);

    assertTrue(aliceAfterInclude > aliceAfterTransfer,
        "Alice gained phantom tokens after re-inclusion");
    console.log("Phantom tokens printed:", (aliceAfterInclude - aliceAfterTransfer)
/ ONE_TOKEN);
}
```

Recommendation

Do not allow `includeInReward` to be called on accounts that have ever held tokens while excluded, or reset the user's `_rOwned` to properly align with the current rate before flipping the boolean.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

YFS-10 | The Auto-Swap Threshold Is Larger Than The Entire Universe

Category	Severity	Location	Status
Design Issue	Medium		Acknowledged

Description

The contract is designed to automatically sell accumulated tax tokens (`_liquidityFee`) to fund a marketing wallet and a buyback treasury. This swap is triggered when the contract's token balance reaches `minimumTokensBeforeSwap The` minimumTokensBeforeSwap` threshold is hardcoded to a value that is larger than the absolute maximum supply of the token. Consequently, the contract will never accumulate enough tokens to trigger a swap, meaning marketing will never be funded and the buyback mechanism is permanently dead. The contract defines the total supply and the swap threshold as follows:

```
uint256 private _tTotal = 19999 * 10**0 * 10**9; // Total Supply: 19,999
uint256 private minimumTokensBeforeSwap = 500 * 10**2 * 10**9; // Threshold: 50,000
```

Because the total supply of the entire token is `19,999` , it is mathematically impossible for the contract's own balance to ever reach `50,000` . The condition `if (contractTokenBalance >= minimumTokensBeforeSwap)` inside `_transfer` will always evaluate to false

Thus, the core economic engine of the token fails on deployment. Marketing funds cannot be generated, the buyback function cannot operate, and the 9% liquidity tax collected on every trade will be permanently locked inside the contract as dead tokens.

Recommendation

Update the hardcoded initialization of `minimumTokensBeforeSwap` to a realistic fraction of the `19,999` total supply (e.g., `50 * 10**9`), or ensure the deployer immediately calls `setNumTokensSellToAddToLiquidity()` with a corrected value before renouncing ownership.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

YFS-11 | Anti-Whale Limit Is Cannot Be Used

Category	Severity	Location	Status
Design Issue	Medium	YfsxCoin.sol	Acknowledged

Description

To prevent market manipulation and massive dumps, the contract implements a `_maxTxAmount` variable, restricting the maximum number of tokens a user can transfer in a single transaction. The `_maxTxAmount` limit set during the `afterPreSale()` public launch function is mathematically larger than the entire total supply of the token, providing zero anti-whale protection. In the `afterPreSale()` function, which is called to finalize the launch and enable strict trading rules, the limit is set as follows:

```
_maxTxAmount = 3000 * 10**2 * 10**9;
```

Because the total supply is only `19,999` tokens, a single user could buy or sell 100% of the entire circulating supply in one atomic transaction without ever hitting the `300,000` limit. The check `require(amount <= _maxTxAmount)` is functionally useless. Thus, the token is fully exposed to whale manipulation, flash loan attacks, and massive price dumps that the community assumes are prevented by the smart contract's code.

Recommendation

Correct the magnitude scaling in `afterPreSale()` so that `_maxTxAmount` represents a logical percentage of the 19,999 supply.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

YFS-12 | Router Should Not Be In The Fee Exempt Whitelist

Category	Severity	Location	Status
Coding Issue	Medium	VulgarTycoon.sol: 258	Acknowledged

Description

Some restrictions can be bypassed by swapping tokens to the router and then getting tokens from the router. When the tokens are transferred to the router, the attacker uses the feature of the router function `removeLiquidityETHSupportingFeeOnTransferTokens`, which will send all the specified tokens in the router to a specified address. Because the router is not supposed to receive tokens for any user other than remove liquidity..

Recommendation

We recommend not to add router to the whitelist.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

YFS-13 | Out Of Gas Exceptions In `difidendToHolders` Loop

Category	Severity	Location	Status
Volatile Code	Medium	VulgarTycoon.sol: 357~364	Acknowledged

Description

When `feeTodifidend` is set above 0, the contract distributes a percentage of the transfer amount as a dividend to existing token holders. This is handled by the `difidendToHolders` function, which iterates over an array of all known holders.

The contract iterates over an unbounded array of all addresses that have ever received tokens to distribute dividends. As the number of unique addresses interacting with the token grows, the gas required to execute a buy or sell transaction will eventually exceed the blockchain's block gas limit, permanently freezing the token.

In the `_transfer` function, any address that receives tokens (excluding the pair, the contract itself, and the zero address) is appended to the `tokenHolders` array:

```
if( recipient != address(0) && !_holderIsExist[recipient] && recipient !=
address(_pairAddress) && recipient != address(this)) {
        tokenHolders.push(recipient);
        _holderIsExist[recipient] = true;
}
```

Addresses are never removed from this array, even if their balance drops to 0. When a dividend is triggered, the contract loops through the entire array:

```
function difidendToHolders(address sender,uint amount) private {
    for(uint256 i = 0; i < tokenHolders.length; i++){ ... }
}
```

Loops over unbounded arrays are a known anti-pattern in Solidity. On the Binance Smart Chain, the block gas limit is finite (approx. 140M gas). Every additional holder adds permanent gas overhead to every single buy and sell.

Once a few thousand unique wallets have traded the token, the cost to execute `_transfer` will exceed the block gas limit. All transactions (buys, sells, and wallet transfers) will revert with an "Out of Gas" error, permanently locking all liquidity in PancakeSwap and all tokens in user wallets.

Recommendation

Abandon the unbounded array loop design. Implement a "pull" dividend model (such as the Dividend-Paying Token standard) where dividends accumulate globally and users must explicitly call a function to claim their share, or use an off-chain distributor.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

YFS-14 | Massive Tax Evasion Via Alternative Pools

Category	Severity	Location	Status
Financial Manipulation	Medium	VulgarTycoon.sol: 251~254, 314~319	Acknowledged

Description

The contract charges a 4% tax on DEX buys and sells, but only a 2% tax on wallet-to-wallet transfers. To differentiate between a DEX trade and a wallet transfer, it checks if the sender or recipient is the `_pairAddress`.

The `_pairAddress` is hardcoded in the constructor to exclusively track the original VIN/WBNB PancakeSwap V2 pool. If a user or whale creates a new liquidity pool on a DEX (e.g., VIN/USDT, or a PancakeSwap V3 pool), the contract will treat trades on that new pool as standard wallet transfers, effectively evading half of the token's taxes.

In the constructor, the pair is set permanently:

```
_pairAddress = IPancakeFactory(_router.factory()).createPair(address(this),
_router.WETH());
```

In `_transfer`, DEX trades are identified strictly by this one variable:

```
if(sender == _pairAddress && !_exemptFee[recipient]) { // buy
// ...
} else if(recipient == _pairAddress && !_exemptFee[sender]) { // sell
```

If a user creates a new pair (e.g. VIN/USDT) and sells their tokens into it, `recipient` will equal the new pool's address, not `_pairAddress`. The contract defaults to the `else if(!_exemptFee[sender] && !_exemptFee[recipient])` block, applying only the 1% burn and 1% distributor fee, completely bypassing the liquidity and LP dividend fees.

Whales can legally evade 50% of the token's tax burden, draining liquidity and bypassing the LP dividend funding mechanism. Furthermore, this completely prevents the project from ever safely listing on a centralized exchange, upgrading to PancakeSwap V3, or bridging to other DEXs, as those external exchanges cannot be taxed correctly.

Scenario

1. Bob, a whale, owns a massive amount of VIN. He wants to sell, but he doesn't want to pay the 4% sell tax.
2. Bob goes to PancakeSwap and clicks "Create New Pool".
3. Instead of pairing his VIN with WBNB, he pairs his VIN with USDT.
4. PancakeSwap creates a brand new address for this VIN/USDT pool. Let's call it Pool_B.
5. Bob sells all his VIN into Pool_B.
6. The VulgarTycoon contract looks at the trade. It sees tokens going to Pool_B. The contract says, "Well, Pool_B is not _pairAddress. This must just be a normal wallet-to-wallet transfer between friends!"

7. The contract bypasses the 4% sell tax and only applies the standard 2% transfer tax.

Recommendation

Change `_pairAddress` to an `isPair` mapping (e.g., `mapping(address => bool) public isPair;`). Provide an `onlyOwner` function allowing the admin to add or remove addresses from the `isPair` mapping so that future pools and exchanges can be taxed appropriately.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

YFS-15 | Rate Reset And Balance Wipeout Via Deliver Function

Category	Severity	Location	Status
Volatile Code	Medium	YfsxCoin.sol: 568	Acknowledged

Description

In YfsxCoin.sol, deliver(uint256 tAmount) lets any non-excluded holder reduce _rTotal by rAmount without changing _tTotal. When the owner has excluded a large passive address from rewards, that excluded address can keep fixed (_rOwned, _tOwned) values while deliver() continues shrinking _rTotal, allowing _getCurrentSupply() to reach its fallback branch. This creates a real accounting edge case that can sharply distort balanceOf results for non-excluded holders.

```
568     function deliver(uint256 tAmount) public {
569         address sender = _msgSender();
570         require(!_isExcluded[sender],
"Excluded addresses cannot call this function");
571         (uint256 rAmount,,,,,) = _getValues(tAmount);
572         _rOwned[sender] = _rOwned[sender].sub(rAmount);
573         _rTotal = _rTotal.sub(rAmount);
574         _tFeeTotal = _tFeeTotal.add(tAmount);
575     }
```

Recommendation

Restrict or remove deliver() so it cannot reduce _rTotal in ways that destabilize reflection accounting.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

YFS-16 | Third-Party Dependency Usage

Category	Severity	Location	Status
Design Issue	Minor	VulgarTycoon.sol: 221; YfsxCoin.sol: 452, 453	Acknowledged

Description

The contract is serving as the underlying entity to interact with one or more third party protocols. The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised and this may lead to lost or stolen assets. In addition, upgrades of third parties can possibly create severe impacts, such as increasing fees of third parties, migrating to new LP pools, etc.

Recommendation

The auditors understood that the business logic requires interaction with third parties. It is recommended for the team to constantly monitor the statuses of third parties to mitigate the side effects when unexpected activities are observed.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

YFS-17 | No Cap On Fees

Category	Severity	Location	Status
Logical Issue	Minor	YfsxCoin.sol: 885, 889	Acknowledged

Description

The `_taxFee` variables in the contract have no set limits, allowing fees to potentially reach 100%. If this occurs, users would receive no tokens from the contract, resulting in a complete loss of their investment.

Recommendation

We recommend setting a reasonable cap on fees and providing adequate disclosure to the community.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

YFS-18 | Zero-Value Transfer Forces Swap Execution

Category	Severity	Location	Status
Design Issue	Minor	VulgarTycoon.sol: 304~305	Acknowledged

Description

The internal swap mechanism (`swapAndDifidend`) is designed to trigger automatically during a normal user sell transaction once the contract has accumulated enough tax tokens.

The `_transfer` function does not require the transfer amount to be greater than zero. An attacker or MEV bot can submit a 0-token transfer directly to the PancakeSwap pair at the exact moment the contract threshold is met, allowing them to perfectly time and trigger the vulnerable `swapAndDifidend` function without exposing any of their own capital.

There is no `require(amount > 0)` check in the transfer logic. If an MEV bot detects that the contract's `_balances[address(this)]` has reached `amountToLPDifidend` (e.g. 5,000 tokens), it can construct a sandwich attack block. In the middle of the sandwich, instead of waiting for a real user to sell, the bot can simply call `token.transfer(_pairAddress, 0)` . Because `recipient == _pairAddress` , the contract routes the call to `processFee` , which proceeds to call `swapAndDifidend(sender)` . The 0-token transfer succeeds, and the contract is forced to dump its 5,000 tokens into the bot's trap.

This allows MEV bots to fully automate the extraction of the LP Dividend funds without requiring organic user volume to trigger the swap. It drastically increases the exploitability of the slippage vulnerability described above.

Recommendation

Add a requirement at the beginning of the `_transfer` function to ensure transfers cannot be empty: `require(amount > 0, "Transfer amount must be greater than zero");` .

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

YFS-19 | Uncallable Setters Due To Private Visibility

Category	Severity	Location	Status
Coding Issue	Minor	VulgarTycoon.sol: 390~398, 402~410	Acknowledged

Description

The contract includes several owner-only setter functions designed to allow the project team to adjust fee percentages dynamically after deployment (e.g., `setBuyFeeToLiquidity`, `setBuyFeeToDifidend`).

The code erroneously declared 6 out of the 10 fee setter functions as `private`. Because they are private and are never called internally by the contract, it is mathematically impossible for anyone, including the owner, to ever change these fees.

The following setters are defined with the `private` visibility modifier:

```
function setBuyFeeToLiquidity(uint256 value) private onlyOwner { ... }
function setBuyFeeToDifidend(uint256 value) private onlyOwner { ... }
function setBuyFeeToLPDifidend(uint256 value) private onlyOwner { ... }
function setSellFeeToLiquidity(uint256 value) private onlyOwner { ... }
function setSellFeeToDifidend(uint256 value) private onlyOwner { ... }
function setSellFeeToLPDifidend(uint256 value) private onlyOwner { ... }
```

A `private` function can only be invoked by other functions residing within the exact same contract. Since no internal functions call these setters, they represent completely dead code.

The token's fee structure is permanently locked into its initial state. The `buyFeeToDifidend` and `sellFeeToDifidend` fees start at `0` and can never be turned on, meaning the core "Holder Dividend" feature is permanently broken for DEX trades. Conversely, the liquidity and LP dividend fees can never be turned off, even if a critical bug requires them to be disabled.

Recommendation

Change the visibility modifier of all setter functions from `private` to `public` or `external` with correct access control checks.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

YFS-20 Stale Fee Cache In Fee-Exempt Transfers Re-Enables Global Fees After Zero-Fee Mode

Category	Severity	Location	Status
Volatile Code	Minor	YfsxCoin.sol: 857~865	Acknowledged

Description

In YfsxCoin.sol, fee-exempt transfers use _tokenTransfer() to temporarily disable fees with removeAllFee() and then restore them with restoreAllFee(). This flow is broken when the active fees are already zero. In that case, removeAllFee() does not refresh the cached values _previousTaxFee and _previousLiquidityFee, but restoreAllFee() still writes those old values back into _taxFee and _liquidityFee. As a result, a fee-exempt transfer can silently turn a legitimate global zero-fee mode back into a non-zero fee mode for the whole token.

```
857     function removeAllFee() private {
858         if(_taxFee == 0 && _liquidityFee == 0) return;
859
860         _previousTaxFee = _taxFee;
861         _previousLiquidityFee = _liquidityFee;
862
863         _taxFee = 0;
864         _liquidityFee = 0;
865     }
```

Recommendation

Update the fee toggle flow so that temporary fee removal cannot restore stale values.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

OPTIMIZATIONS | YFSX.VIN

ID	Title	Category	Severity	Status
YFS-21	Lack Of Array Bounds On Excluded Accounts	Gas Optimization	Optimization	Acknowledged
YFS-22	`amountToStopBurn` Is A Ghost Variable	Code Optimization	Optimization	Acknowledged

YFS-21 | Lack Of Array Bounds On Excluded Accounts

Category	Severity	Location	Status
Gas Optimization	Optimization	YfsxCoin.sol	Acknowledged

Description

The contract maintains an array `_excluded` containing the addresses of all users who have been excluded from earning reflections via `excludeFromReward. The` _getCurrentSupply() `function iterates over the entire` _excluded `array on every single token transfer. Because there is no maximum limit on the array's size, excluding too many accounts will cause the loop to run out of gas, permanently freezing all trading. In` _getCurrentSupply()`:

```
for (uint256 i = 0; i < _excluded.length; i++) {
    // ... math operations ...
}
```

Since this function is called inside `_getRate()`, which is called inside `_getValues()`, which is called on *every single transfer*, the gas cost of a standard ERC-20 transfer increases every time a new address is excluded. If the array grows too large, the gas required to execute a simple transfer will exceed the network's block gas limit.

Recommendation

Implement a strict `require(_excluded.length < MAX_EXCLUDED)` check inside `excludeFromReward()` to ensure the array can never grow large enough to cause an Out-of-Gas revert.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

YFS-22 | `amountToStopBurn` Is A Ghost Variable

Category	Severity	Location	Status
Code Optimization	Optimization	YfsxCoin.sol: 231	Acknowledged

Description

Deflationary tokens often include a threshold to stop burning tokens once the total supply reaches a certain minimum target. The contract defines an `amountToStopBurn` variable, initialized in the constructor to 50% of the initial supply. The `amountToStopBurn` variable is declared, initialized, and even given a setter function, but it is never actually referenced in the contract's transfer or fee logic. The burn mechanism will never stop, regardless of how small the supply gets.

In the constructor:

```
amountToStopBurn = _totalSupply.div(2);
```

However, neither the `processFee` function nor the `_transfer` function includes any conditional logic to check `if (_totalSupply <= amountToStopBurn)`. The 1% burn fee is applied unconditionally on every taxable transfer.

Recommendation

Implement the missing logic check in the burn calculation. For example, before taking the burn fee, check `if (_totalSupply > amountToStopBurn)`. If it is less than or equal to the target, the `burnAmount` should be forced to `0`.

Alleviation

[YFSX.VIN, 04/10/2026]: Because the permissions were revoked 4 years ago, there is no function to operate tokens.

APPENDIX | YFSX.VIN

Finding Categories

Categories	Description
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Financial Manipulation	Financial Manipulation findings indicate issues in design that may lead to financial losses.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Gas Optimization	Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction on the total gas cost of a transaction.
Code Optimization	No description available.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.